Parcelo: Secure Coding Practices

Start here: what this file is, in plain English

What is "secure coding"? It is a set of habits that stop people from breaking, cheating or spying on your system. This file lists those habits for Parcelo, with a short "In plain words" note under each section so you don't need a technical background to follow it.

The one idea behind everything. Think of Parcelo as a shop.

- The **website** (the React apps) is the shop window. Anyone can walk up to it, and some people will try to mess with it. So you can never trust what the website says. If it says "this delivery costs N100" or "I am the admin", the system must not just believe it.
- The **backend** (Firebase: Cloud Functions and security rules) is the back room. Customers can't enter it. All the real checking happens there.

So the golden rule is: **the website asks, the backend decides.**

Who it's for. You, and also any AI assistant or developer building the project. When you give this file to a coding assistant, it knows what "safe" means for Parcelo.

How to read it. You don't have to understand every technical line. Read the "In plain words" note under each section, and look up any unfamiliar word in the glossary below.

Glossary (words you'll see)

Word	What it means
Backend	The hidden part of the system that stores data and applies the rules. For Parcelo this is Firebase.
Frontend	The part people see and click: your React websites.
Authentication	Proving who you are (logging in).
Authorization	Deciding what you are allowed to do once we know who you are.
Role	A label on a user: customer, admin, dispatcher or driver. It decides what they can do.
Custom claim	A role label stored safely inside a user's login, which only the backend can set. Users can't edit it.
Firestore	The database where orders, shipments and users are stored.
Security rules	A rulebook attached to the database and file storage that says who can read or write what. It's the last line of defence.
Cloud Function	A small program that runs in the backend (not in the browser). Booking, changing status and similar actions run here.
Callable function	A Cloud Function your website can call by name, like pressing a button that asks the backend to do something.
Transaction	A group of changes done "all or nothing". For example, creating an order, its shipment and its tracking record together. If one part fails, none are saved.
Validation	Checking that input is sensible (for example weight isn't negative, a name isn't 5,000 letters).
Schema (Zod)	A written description of what valid input looks like, used to do validation automatically.
App Check	A Firebase feature that makes sure requests come from your real apps and not from someone's script.
Rate limiting	Slowing down anyone who sends too many requests too quickly.
Brute force / enumeration	Trying thousands of guesses (for example tracking numbers) hoping some work.
PII	Personal information: names, phone numbers, addresses, emails.
EXIF	Hidden details inside a photo, such as where it was taken (GPS).
CSP / security headers	Settings sent with your website that tell browsers to block risky behaviour.

Secret / API key	A private password a service gives you (for SMS, email, payments). It must never appear in website code.
Emulator	A copy of Firebase that runs on your computer so you can test safely.
Dependency	Someone else's code library that your project uses.

The 8 most important points, in one list

1. The server sets the price, tracking number and status. The browser never does.
2. Roles can only be given by the backend, and public sign-up creates customers only.
3. Everything is blocked by default; you open only what each role needs.
4. A customer can only ever see their own orders.
5. Tracking numbers must be long and random so nobody can guess them.
6. The public tracking page shows no names, phone numbers or full addresses.
7. Passwords and API keys never go in website code or in git.
8. Test the rules before launch (the checklist in section 13).

Applies to: React (Vite) apps, Firebase Auth, Firestore, Storage, Cloud Functions (Node.js), Firebase Hosting. Core rule: **the browser is untrusted. Every real decision (who you are, what you may do, what a price is, what a status can become) is made in Cloud Functions or security rules.**

---

1. Ground rules

> **In plain words:** These are the six basic principles. Everything else in the file is these principles applied to a specific part of the project. If you only remember one thing: never let the website be the one that decides anything important.

1. **Default deny.** Firestore and Storage rules start with `allow read, write: if false;` and open only what each role needs.
2. **Least privilege.** Each role gets the minimum collections, fields and actions (see blueprint section 2.6).
3. **Never trust the client.** Hiding a button or route (RoleGuard) is cosmetic. The rules and functions are the protection.
4. **Server decides sensitive values.** Price, tracking number, status, payment status, timestamps and `changedBy` are set only in Cloud Functions.
5. **Validate at every boundary.** Inputs to functions, rule conditions on writes, and form validation in the UI (UI validation is for users, not security).
6. **Fail closed, say little.** On error, deny and return a generic message. Never return stack traces or internal ids.

---

2. Authentication and roles

> **In plain words:** Logging in proves who someone is. Their role decides what they may do. Firebase handles passwords for you (don't build your own). The role is stored in the login itself as a "custom claim", and only the backend can set it. That's why a customer can't just turn themselves into an admin. The very first admin is created by hand by you, once, in the Firebase console. After that, only an admin can create other staff.

- Use Firebase Authentication (email + password). Never store or handle passwords yourself.
- **Roles live in custom claims** (`customer` , `admin` , `dispatcher` , `driver`). Set them only from a Cloud Function using the Admin SDK (`setCustomUserClaims`).
- Public registration creates **customers only**. The `role` is never read from the request body. Set `customer` server-side (an `onCreate` auth trigger or a `registerCustomer` function).
- Create the first admin manually (Firebase console or a one-off script run locally with a service account). Never ship an "make me admin" function.
- After a role change, revoke refresh tokens (`revokeRefreshTokens`) and force a token refresh on the client (`getIdToken(true)`).
- Do **not** trust a `role` field stored in the `users` document for authorization. Use `request.auth.token.role` in rules and `context.auth.token.role` in functions. If a `users.role` copy exists, treat it as display only and make it unwritable by clients.
- Enforce password strength (min 8 chars; Firebase default is 6, so raise it with the Identity Platform policy or check in

your register flow). Enable email enumeration protection in Authentication settings.
- Enable **Firebase App Check** (reCAPTCHA Enterprise or v3 for web) and enforce it on Firestore, Storage and callable functions to block scripts that are not your apps.
- Wrong-app logins: show the WrongAppNotice, but the data is still protected by rules, not by which app you opened.

---

3. Firestore security rules

> **In plain words:** The security rules are a rulebook sitting on the database itself. Even if someone bypasses your website and talks to the database directly, the rulebook still blocks them. The approach is: block everything first, then allow only what's needed. Customers can read their own orders but can't write to them at all (the backend writes orders for them). The public tracking records can be fetched one at a time by anyone who has the number, but nobody can list or edit them. The tests at the end of this section are important: they prove the rules really work, for example that customer A can't open customer B's order.

Helper functions (top of `firestore.rules`):

```
function signedIn() { return request.auth != null; }
function role() { return request.auth.token.role; }
function isAdmin() { return signedIn() && role() == "admin"; }
function isStaff() { return signedIn() && (role() == "admin" || role() == "dispatcher"); }
function isOwner(uid) { return signedIn() && request.auth.uid == uid; }
```

Rules per collection (Phase 1):

Collection	Read	Write
`users/{uid}`	owner or admin	owner may update only safe fields (`name` , `phone`); never `role`
`quotes`	owner (`customerId`)	none (functions only)
`orders`	owner or staff	none (functions only)
`shipments` + `events`	staff; owner via their order (copy `customerId` onto shipment so rules can check it without a join)	none directly, except admin rider fields (below)
`publicTracking/{trackingNumber}`	`get` only, anyone	none
`settings`	admin (pricing is read by `getQuote` server-side)	admin only

Specific requirements:

- **No client write to `orders` , `quotes` , `publicTracking` , `shipments.status` , `paymentStatus` .** All via functions.
- **Rider fields** (admin writes directly in Phase 1): restrict with `request.resource.data.diff(resource.data).affectedKeys().hasOnly(["riderName","riderPhone"])` and validate types and length (`is string` , `size() <= 80`). This prevents an admin-side bug or stolen admin session from rewriting status through this path.
- **`publicTracking` :** `allow get: if true; allow list, write: if false;` Keep it free of names, phones, full addresses and any internal ids that map to other collections.
- Validate shape on any allowed write: required keys, types, string lengths, numeric ranges.
- Never use `allow read: if true` on a collection other than `publicTracking` .
- **Test the rules with the Firebase Emulator** (`@firebase/rules-unit-testing`). Required tests: customer A cannot read customer B's order; customer cannot write any order/shipment; unauthenticated cannot list `publicTracking` ; a user cannot self-assign `role` .

---

4. Cloud Functions

> **In plain words:** Cloud Functions are the back-room workers. Each time the website asks for something (get a quote, book a delivery, change a status), the worker does the same careful routine: (1) Who are you? (2) Are you allowed? (3) Is what you sent sensible? (4) Is this action allowed right now (for example, a package can't jump from Pending

> straight to Delivered)? (5) Do all the saving together, all or nothing. (6) Reply with only what's needed. The function list below shows the special checks for each action. A key example is booking: the server recalculates the price itself and ignores whatever price the browser shows, so nobody can edit the price in their browser and pay less.

Every callable function follows this order:

1. **Authenticate**: `if (!context.auth) throw new HttpsError("unauthenticated", ...)`.
2. **Authorize**: check `context.auth.token.role` against the allowed roles for that function.
3. **Validate input** with a schema library (e.g. Zod). Reject unknown fields, wrong types, oversized strings, negative or absurd numbers (`weightKg` 0.1 to 1000; `size` in an enum).
4. **Check state** (status transition allowed by `statusRules`, quote not expired, quote belongs to caller).
5. **Write inside a Firestore transaction or batch** so order + shipment + first event + public record are created all-or-nothing.
6. **Return minimal data** and a safe error.

Function-specific rules:

- **`getQuote`**: compute price from `settings` on the server; geocode/distance server-side or validate client-supplied coordinates (range check lat -90..90, lng -180..180). Store the quote with `customerId` and `expiresAt`.
- **`bookDelivery`**: accept only `quoteId` and `paymentMethod` (enum). Load the quote, confirm `quote.customerId == uid` and not expired, **recompute the price** and compare, mark the quote used (prevents double-booking from one quote, which also gives you idempotency).
- **`updateShipmentStatus`**: enforce `statusRules` on the server; Delivered requires a saved proof; Failed requires a reason (length-limited); record `changedBy = uid` and server timestamp; update `shipments`, `orders` and `publicTracking` in one transaction.
- **`cancelOrder`**: only the owner, only while Pending.
- **`setPaymentStatus`**: admin only; allowed transitions only.
- **`saveProof`**: verify the Storage object exists at the expected path, is an image, and under the size limit before saving its reference.
- **`createStaffAccount` / `setUserRole`** (Phase 2): admin only, role from an allow-list enum, log who created whom.
- Set a sensible `maxInstances`, timeouts and memory. Enable per-user rate limiting for `getQuote` and the public tracking lookup (App Check + a simple counter in Firestore or a rate-limit library).
- Use `HttpsError` codes from the blueprint table. Log the detail server-side (without PII); return a short message to the client.
- Pin a Node LTS runtime and keep `firebase-functions` and `firebase-admin` current.

---

5. Tracking numbers and the public lookup

> **In plain words:** Anyone can type a tracking number without logging in. That's convenient, but it means a stranger could try thousands of numbers to peek at other people's deliveries. Two defences: (1) make numbers long and random so guessing is hopeless (an 8-digit number like the one in your blueprint example is too easy to guess), and (2) show only harmless details on the public page (status, area, timeline). No names, phone numbers or full addresses. Also pick one format (`TRK-` followed by letters/numbers) and use it everywhere, because your blueprint currently shows two formats.

- The blueprint example (`TRK-84729301`, 8 digits) is **guessable by brute force**. Generate tracking numbers with `crypto.randomBytes` (not `Math.random`), e.g. `TRK-` + 10 to 12 characters from an unambiguous alphabet (no 0/O/1/I), ~50+ bits of entropy.
- Use the tracking number as the `publicTracking` document id and check for collisions in the transaction.
- Keep one canonical format. The blueprint shows both `TRK-84729301` and `TRK84729301`; pick one, validate it with a regex in the function and the UI, and normalise (uppercase, trim) before lookup.
- Public record contains only: tracking number, status, destination **area** (e.g. "Lekki, Lagos"), timeline (status + time). No names, phones, street addresses, order/shipment ids or driver data (Phase 2).
- App Check enforcement plus the `get`-only rule stops listing; enable throttling to slow enumeration. Return the same friendly "not found" message for malformed and unknown numbers.
- Proof-of-delivery photos shown on the public page may contain a person's home or face. Serve via a long unguessable path or short-lived signed URL, and show it only once Delivered.

---

6. Cloud Storage and uploads

> **In plain words:** Proof-of-delivery photos are files uploaded by users, and files can be dangerous or huge. So: only the right people can upload, only JPG or PNG, under 2 MB, with random file names. Phone photos secretly record where they were taken, so we strip that hidden data. A delivery photo can also show someone's home or face, so don't make photos publicly browsable.

- Rules: write allowed only to `proofs/{shipmentId}/...` and only for admin (Phase 1) or the assigned driver (Phase 2). Enforce `request.resource.size < 2 * 1024 * 1024` and `request.resource.contentType.matches('image/(jpeg|png)')`.
- Never trust file extension or client-supplied content type alone; the rule checks `contentType`, and `saveProof` re-checks metadata.
- Generate filenames server-side or with a random id; never use the user's filename in a path.
- Compress client-side (`imageCompress.js`) and **strip EXIF** (it can contain GPS coordinates).
- No public listing. Read access only for staff and the order owner, or tokenised URLs for the public page as above.

---

7. Front-end (React)

> **In plain words:** Even though the website isn't trusted to make decisions, you still build it carefully. Don't insert raw text as HTML (a trick attackers use to run their own code on your page). Don't put any private keys in it, because anyone can open the browser tools and read it. (The Firebase "config" values are fine to be public; they're protected by the rules.) The security headers are instructions to the browser such as "only load things from my own site" and "don't let other websites show my pages inside a frame".

- Never use `dangerouslySetInnerHTML`. React escapes by default; keep it that way for notes, failure reasons, addresses and names.
- Validate and trim all form inputs (`validators.js`) and show friendly errors, but treat this as UX only.
- Build links safely: only `https:` URLs; for `Call` buttons use `tel:` with a number validated by regex.
- No secrets in the bundle. Firebase web config in `.env` (`VITE_*`) is **public by design**; real protection is rules + App Check. Never put service-account keys, Termii/Resend/Paystack keys or OpenRouteService keys in any `VITE_*` variable.
- Do not store tokens in `localStorage`; rely on Firebase Auth's persistence. Keep role/user data in React Context only.
- Don't log tokens, emails, phones or addresses to the console in production builds.
- Add security headers in `firebase.json` for each Hosting site:
- `Content-Security-Policy` (`default-src 'self'`; allow Firebase, OpenStreetMap tile host, Nominatim, Google Fonts if used; no `unsafe-inline` scripts)
- `Strict-Transport-Security: max-age=31536000; includeSubDomains`
- `X-Content-Type-Options: nosniff`
- `X-Frame-Options: DENY` (or CSP `frame-ancestors 'none'`)
- `Referrer-Policy: strict-origin-when-cross-origin`
- `Permissions-Policy` (geolocation only for the Driver app)
- Self-host fonts from `packages/shared/src/theme` (as the blueprint says) so no third-party font request leaks visitor data.
- **Nominatim:** it is a public service with a usage policy (max 1 request/second, identify the app, no autocomplete). Trigger on button press as planned; for production consider calling it from a function with a proper User-Agent/contact, or self-host. Treat returned coordinates as untrusted and range-check them.

---

8. Secrets, config and environments

> **In plain words:** A "secret" is a private password for a service (SMS, email, payments). Keep them in Firebase's secret store, never in your code and never uploaded to GitHub. Use one Firebase project for testing and a separate one for real customers, so a testing mistake can't touch real data. If a key is ever accidentally posted publicly, replace it immediately.

- Use **separate Firebase projects** for dev, staging and production (`.firebaserc` aliases).
- Function secrets (Phase 2/3: OpenRouteService, Termii, Resend, payment keys) go in **Secret Manager** via `defineSecret`, never in code, `.env` committed files or logs.
- `.gitignore` : `.env*`, service-account JSON, `firebase-debug.log` , `.runtimeconfig.json` . Add a pre-commit secret scan (e.g. gitleaks).
- Rotate any key that was ever committed.
- Restrict the browser API key in Google Cloud console to your three Hosting domains.

---

9. Data protection and privacy

> **In plain words:** You'll hold people's names, phone numbers and addresses, and in Nigeria there are data protection laws about that (the NDPA). Collect only what you need, don't copy it around, don't put it in logs, and let people delete their account. For drivers, only share their location while they're actually delivering, and delete old location points after a day.

- Collect only what is needed. Phone numbers and addresses are personal data (Nigeria Data Protection Act / NDPR applies); state purpose in a short privacy notice and allow account deletion.
- Copy minimal denormalised data into orders (name, area); don't scatter phone numbers across collections.
- Logs and the notifications log (Phase 3) must not contain full addresses or message bodies with personal details.
- Driver location (Phase 2): only while Out for Delivery; delete pings after 24 hours (scheduled function); tell the driver when sharing is on.
- Backups: enable scheduled Firestore exports and test a restore.

---

10. Dependencies and supply chain

> **In plain words:** Your project uses code libraries written by other people. Sometimes a library has a security hole or gets tampered with. Keep the list short, run `npm audit` (a built-in check that flags known problems), and keep libraries updated.

- Commit lockfiles; install with `npm ci` in CI.
- Run `npm audit` and enable Dependabot/Renovate. Review new packages (downloads, maintainers) before adding.
- Keep the dependency list small. Prefer Firebase's own SDKs.
- Pin GitHub Actions to commit SHAs if you use CI; give the deploy token only deploy permissions.

---

11. Logging, monitoring and audit

> **In plain words:** Keep a diary of important events (who changed which status, when). That diary can't be edited afterwards, so if there's a dispute you can see exactly what happened. Set alerts for strange activity (many failed logins) and a spending alert so a bug or attack can't create a surprise bill.

- Every status change, assignment, payment change and role change writes an event with `changedBy` and server time (already in the blueprint). Make events append-only (rules deny update/delete).
- Use Cloud Logging alerts for spikes in `permission-denied` , failed logins and tracking lookups.
- Set Firebase budget alerts (a runaway loop or abuse should not become a surprise bill).

---

12. Phase 2 and 3 forward notes

> **In plain words:** You don't need this section for Phase 1. It's a heads-up for later. For example, when you add drivers, a driver must only see their own jobs; when you add online payments, only trust a payment once the payment

> company confirms it to your server (never because the customer's browser says "I paid"); and assigning a driver must be all-or-nothing so two dispatchers can't give the same driver two jobs.

- **Driver:** rules allow updating only `lastLocation` on their own record (as in blueprint). Drivers read only shipments where `driverId == uid`; status changes via `updateShipmentStatus`, which checks assignment and one-step moves.
- **Assignment:** do it in a Firestore transaction (check shipment Pending, driver and vehicle available, then write), which stops double-booking.
- **Customer live view:** `copyDriverToShipment` shares first name, plate and position only while Out for Delivery; clear it afterwards.
- **Payments (optional):** mark Paid only from a verified webhook (check the provider signature, amount and reference match); never from a client redirect. Card data never touches your code.
- **Notifications:** retries with caps, no personal data beyond what's needed, honour opt-outs for optional messages.
- **Inventory:** stock decrement inside the same transaction as order creation; floor at zero.

---

13. Pre-release security checklist (Phase 1)

> **In plain words:** Before you show Parcelo to real users, go through this list and tick every box. If you can't tick one, fix it first. Each box is a quick test or check, for example "log in as customer A and try to open customer B's order; it must fail".

- ☐ Rules deny by default; emulator tests pass (cross-customer read, direct writes, listing `publicTracking`, role self-escalation)
- ☐ Only the register function can create users, and only as `customer`
- ☐ All callable functions authenticate, authorize, validate with a schema, and use transactions
- ☐ Price recomputed server-side; quote bound to the caller, expiring, single-use
- ☐ Tracking numbers are random, 10+ chars, one canonical format
- ☐ Public tracking returns no PII; rate-limited; App Check on
- ☐ Storage rules enforce path, JPG/PNG, under 2 MB; EXIF stripped
- ☐ Delivered refused without proof; Failed requires a reason
- ☐ No secrets in `VITE_*` or git; API key restricted by domain
- ☐ Security headers and CSP set on all hosting sites
- ☐ `npm audit` clean (or accepted risks documented)
- ☐ Separate dev/prod Firebase projects; budget alert set